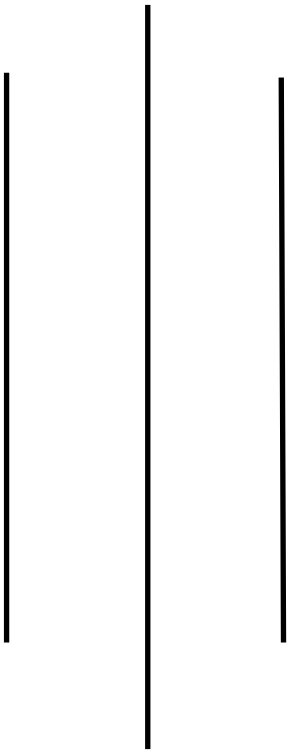
LAPORAN UAS KOMPUTER GRAFIK

Dosen Pengampu :

Teuku Risky Noviandi, S.KOM., M.KOM



NAMA : RIANA

NIM : 24146012

MK : Komputer Grafik

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS ABULYATAMA

CODE PYTHON :

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

# ================= KUBUS 3D =================
cube_vertices = [
   (-1,-1,-1),(1,-1,-1),(1,1,-1),(-1,1,-1),
   (-1,-1,1),(1,-1,1),(1,1,1),(-1,1,1)
]

cube_edges = [
   (0,1),(1,2),(2,3),(3,0),
   (4,5),(5,6),(6,7),(7,4),
   (0,4),(1,5),(2,6),(3,7)
]

cube_pos = [-2,0,-6]
cube_rot_x = 0
cube_rot_y = 0
cube_scale = 1

def draw_cube():
   glEnable(GL_BLEND)
   glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)

   glColor4f(0.6, 0.6, 0.6, 0.5)  # abu transparan
   glBegin(GL_LINES)
   for edge in cube_edges:
      for v in edge:
```

```
            glVertex3fv(cube_vertices[v])
    glEnd()

    glDisable(GL_BLEND)

# ================= PERSEGI 2D =================
square_vertices = [
    (0,0,0),(2,0,0),(2,2,0),(0,2,0)
]

def apply_reflection(vertices, axis=None):
    result = []
    for x,y,z in vertices:
        if axis == 'x':
            result.append((x,-y,z))
        elif axis == 'y':
            result.append((-x,y,z))
        else:
            result.append((x,y,z))
    return result

def apply_shearing(vertices, shear_x=0):
    result = []
    for x,y,z in vertices:
        result.append((x + shear_x*y, y, z))
    return result

sq_pos = [2,0]
sq_rot = 0
sq_scale = 1
shear_x = 0
```

```
reflect_axis = None

def draw_square():
    v = apply_reflection(square_vertices, reflect_axis)
    v = apply_shearing(v, shear_x)

    glColor3f(1.0, 0.8, 0.0)  # KUNING
    glBegin(GL_QUADS)
    for p in v:
        glVertex3fv(p)
    glEnd()

# ================= MAIN =================
pygame.init()
display = (800,600)
pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
pygame.display.set_caption("Transformasi 2D & 3D")

clock = pygame.time.Clock()

while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            quit()
        elif event.type == KEYDOWN:
            if event.key == K_1: shear_x = 1
            if event.key == K_2: shear_x = 0
            if event.key == K_3: reflect_axis = 'x'
            if event.key == K_4: reflect_axis = 'y'
            if event.key == K_5: reflect_axis = None
```

```
keys = pygame.key.get_pressed()

# === KUBUS ===
if keys[K_LEFT]: cube_pos[0] -= 0.1
if keys[K_RIGHT]: cube_pos[0] += 0.1
if keys[K_UP]: cube_pos[1] += 0.1
if keys[K_DOWN]: cube_pos[1] -= 0.1
if keys[K_w]: cube_pos[2] += 0.1
if keys[K_s]: cube_pos[2] -= 0.1
if keys[K_a]: cube_rot_y -= 5
if keys[K_d]: cube_rot_y += 5
if keys[K_q]: cube_rot_x -= 5
if keys[K_e]: cube_rot_x += 5
if keys[K_z]: cube_scale += 0.05
if keys[K_x]: cube_scale = max(0.1, cube_scale - 0.05)

# === PERSEGI ===
if keys[K_i]: sq_pos[1] += 0.1
if keys[K_k]: sq_pos[1] -= 0.1
if keys[K_j]: sq_pos[0] -= 0.1
if keys[K_l]: sq_pos[0] += 0.1
if keys[K_u]: sq_rot += 5
if keys[K_o]: sq_rot -= 5
if keys[K_n]: sq_scale += 0.05
if keys[K_m]: sq_scale = max(0.1, sq_scale - 0.05)

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
glEnable(GL_DEPTH_TEST)

# === KUBUS 3D ===
```

```
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluPerspective(45, display[0]/display[1], 0.1, 50)

glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
glTranslatef(*cube_pos)
glRotatef(cube_rot_x,1,0,0)
glRotatef(cube_rot_y,0,1,0)
glScalef(cube_scale,cube_scale,cube_scale)
draw_cube()

# === PERSEGI 2D ===
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluOrtho2D(-4,6,-4,4)

glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
glTranslatef(sq_pos[0],sq_pos[1],0)
glRotatef(sq_rot,0,0,1)
glScalef(sq_scale,sq_scale,1)
draw_square()

pygame.display.flip()
clock.tick(60)
```

PENJELASAN CODE :

1. Penjelasan Umum Program

Program ini dibuat menggunakan Python, Pygame, dan PyOpenGL untuk menampilkan dan memanipulasi objek 3D (kubus) dan objek 2D (persegi) dalam satu jendela.

Tujuan utama program adalah menerapkan transformasi geometri, yaitu:

- Translasi
- Rotasi
- Skala
- Shearing
- Refleksi

Transformasi 3D diterapkan pada kubus, sedangkan transformasi 2D diterapkan pada persegi.

2. Library yang Digunakan

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *
```

Penjelasan:

- pygame → mengelola window, event keyboard, dan loop program
- OpenGL.GL → fungsi rendering objek OpenGL
- OpenGL.GLU → fungsi bantu kamera (perspective & ortho)

3. Objek Kubus 3D

3.1 Data Titik (Vertices)

```
cube_vertices = [
  (-1,-1,-1),(1,-1,-1),(1,1,-1),(-1,1,-1),
  (-1,-1,1),(1,-1,1),(1,1,1),(-1,1,1)
]
```

- Terdiri dari 8 titik sudut kubus
- Menggunakan koordinat 3D (x, y, z)

- Ukuran kubus simetris terhadap titik pusat (0,0,0)

3.2 Garis Kubus (Edges)

```
cube_edges = [
  (0,1),(1,2),(2,3),(3,0),
  (4,5),(5,6),(6,7),(7,4),
  (0,4),(1,5),(2,6),(3,7)
]
```

- Menentukan hubungan antar titik
- Digambar menggunakan GL_LINES
- Membentuk rangka (wireframe) kubus

3.3 Variabel Transformasi Kubus

```
cube_pos = [-2,0,-6]
cube_rot_x = 0
cube_rot_y = 0
cube_scale = 1
```

- cube_pos → posisi kubus (translasi)
- cube_rot_x, cube_rot_y → sudut rotasi
- cube_scale → skala kubus

3.4 Fungsi draw_cube()

```
def draw_cube():
  glEnable(GL_BLEND)
  glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)

  glColor4f(0.6, 0.6, 0.6, 0.5)
  glBegin(GL_LINES)
  ...
  glEnd()
```

Penjelasan:

- GL_BLEND → mengaktifkan transparansi
- Warna abu-abu transparan
- Kubus digambar sebagai wireframe

4. Objek Persegi 2D

4.1 Titik Persegi

```
square_vertices = [
  (0,0,0),(2,0,0),(2,2,0),(0,2,0)
]
```

- Persegi berada pada bidang 2D (sumbu Z = 0)
- Digambar menggunakan GL_QUADS

4.2 Refleksi

```
def apply_reflection(vertices, axis=None):
```

- Refleksi terhadap:
  - Sumbu X → (x, -y)
  - Sumbu Y → (-x, y)
- Jika axis=None, tidak ada refleksi

4.3 Shearing

```
def apply_shearing(vertices, shear_x=0):
  new_x = x + shear_x * y
```

- Shearing pada sumbu X
- Bentuk persegi menjadi miring

4.4 Fungsi draw_square()

```
glColor3f(1.0, 0.8, 0.0)
```

- Persegi berwarna kuning
- Transformasi refleksi & shearing diterapkan sebelum Digambar

5. Pengaturan Tampilan (Proyeksi)

Kubus 3D

gluPerspective(45, aspect, 0.1, 50)

→ Tampilan perspektif 3D

Persegi 2D

gluOrtho2D(-4,6,-4,4)

→ Tampilan ortografik 2D

6. Kesimpulan

Program ini berhasil menggabungkan:

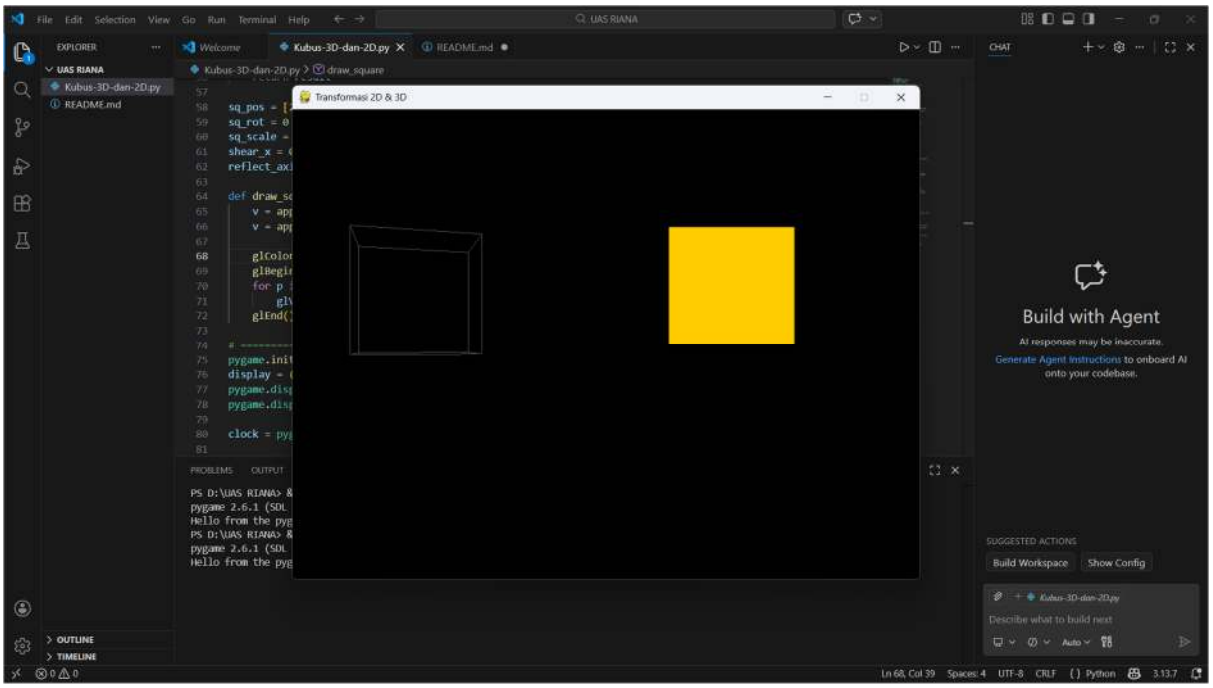
- Transformasi 3D pada kubus
- Transformasi 2D pada persegi
- Sistem kontrol keyboard interaktif
- Penggunaan proyeksi perspektif dan ortografik secara bergantian

Program ini sesuai untuk pembelajaran transformasi geometri komputer grafis.

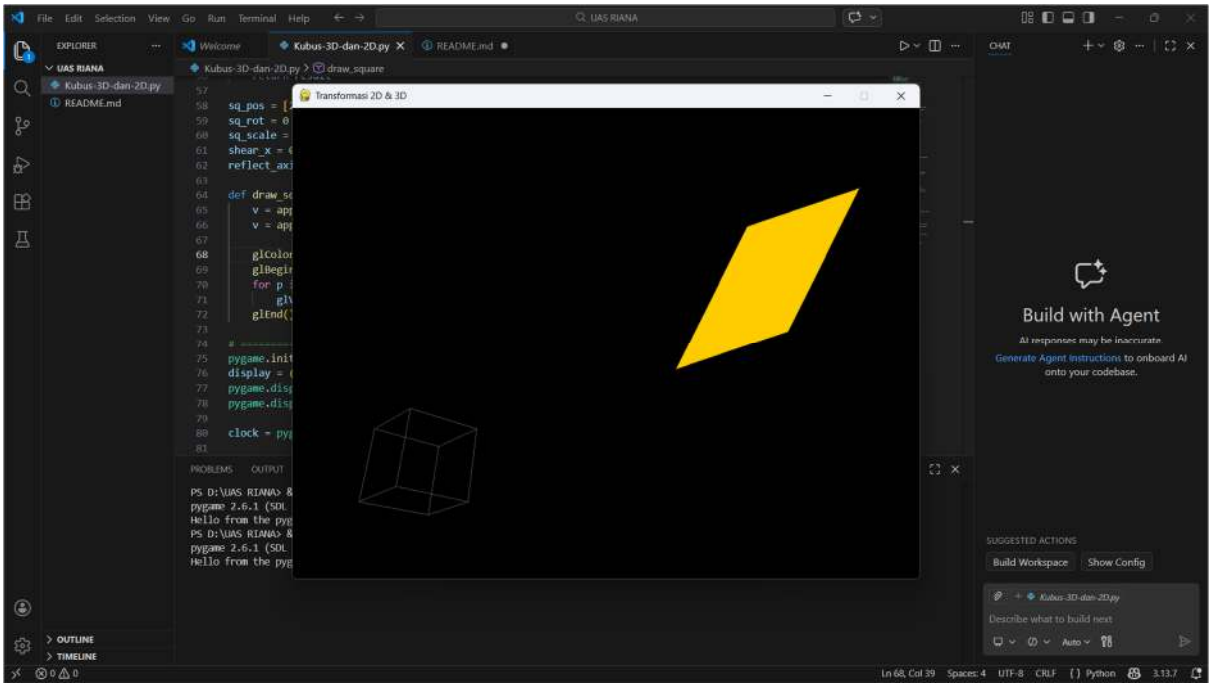
TAMPILAN HASIL KODE :

TRANSLASI, ROTASI, SKALA, SHEARING, DAN REFLEKSI

TAMPILAN AWAL :



TAMPILAN AKHIR :



CARA MENJALANKAN :

Sistem Kontrol Keyboard

Kontrol Kubus 3D

Tombol	Fungsi
← / →	Translasi sumbu X
↑ / ↓	Translasi sumbu Y
W / S	Translasi sumbu Z
A / D	Rotasi sumbu Y
Q / E	Rotasi sumbu X
Z	Perbesar skala
X	Perkecil skala

Kontrol Persegi 2D

Tombol	Fungsi
I / K	Translasi sumbu Y
J / L	Translasi sumbu X
U / O	Rotasi
N	Perbesar skala
M	Perkecil skala

Transformasi Khusus Persegi

Tombol	Fungsi
1	Shearing X aktif
2	Shearing X mati
3	Refleksi sumbu X
4	Refleksi sumbu Y
5	Reset refleksi

Dapat di jalankan menggunakan control keyboard 3D dan 2D yang tertera di atas

LINK GITHUB :

https://github.com/SourcesDVLP/RIANA.git